

BitCPM-CANN: Native 1.58-Bit Large Language Model Training on Ascend NPU

BitCPM Team



<https://github.com/OpenBMB/MiniCPM>



<https://huggingface.co/collections/openbmb/bitcpm-cann>

Abstract

We present BitCPM-CANN, a systematic family-level study of 1.58-bit (ternary) quantization-aware training (QAT) on the Huawei Ascend NPU platform. To address two practical gaps for extreme low-bit LLMs—whether ternary weights preserve capabilities on complex reasoning tasks at on-device scales, and how to make end-to-end 1.58-bit training natively available outside the CUDA ecosystem—we port our prior GPU-based pipeline to CANN, MindSpeed, and Megatron-LM, and train four models (BitCPM-CANN-0.5B/1B/3B/8B) strictly aligned with their full-precision MiniCPM4 counterparts in architecture and pre-training data. Across 11 benchmarks spanning commonsense reasoning, domain knowledge, and mathematics & reasoning, the 1B, 3B, and 8B variants retain 95.7%–97.2% of full-precision performance, with the 3B variant achieving parity on BBH and the 3B/8B variants recovering nearly all of GSM8K. The 0.5B variant retains 90.1%, with the residual gap concentrated on mathematics, indicating that capacity—not the quantizer—is the bottleneck at sub-billion scales. Our QAT integration adds only a 4.5% training throughput overhead (148 vs. 155 TFLOP/s per NPU), making ternary training viable as a default configuration, while enabling up to an $8\times$ weight memory reduction (approximately $6\times$ end-to-end including scaling factors) at inference. To our knowledge, this is the first end-to-end 1.58-bit training system on a domestic NPU scaled up to 8B parameters, providing a reusable low-bit training infrastructure for the Ascend ecosystem.

1 Introduction

Deploying large language models (LLMs) is heavily bottlenecked by memory capacity and bandwidth, a challenge that has been further exacerbated by recent hardware market dynamics. Specifically, DRAM and NAND Flash contract prices have risen sharply due to the shifting of global wafer capacity toward high-bandwidth memory (HBM) for AI servers [TrendForce \(2026\)](#); [IDC \(2026\)](#). This pricing pressure primarily impacts model inference, especially in on-device scenarios where memory budgets are strictly limited by system-on-chip (SoC) architectures and cannot be easily scaled out. Among potential solutions, compressing weight bit-widths offers a direct remedy: with a b -bit weight representation, a given memory capacity can accommodate a $16/b$ -times larger model, longer context lengths, or additional concurrent instances.

Extreme low-bit quantization—most notably the 1.58-bit (ternary) representation where weights are restricted to $\{-1, 0, 1\}$ [Wang et al. \(2023\)](#); [Ma et al. \(2024\)](#)—represents the limit of this approach, theoretically reducing the weight memory footprint by $8\times$ compared to 16-bit floating-point (BF16) formats. Recent scaling law analyses suggest that sub-2-bit representations can be Pareto-optimal in the trade-off between model accuracy and parameter size [Liu et al. \(2025\)](#). Moreover, end-to-end ternary training has been successfully scaled up to the 2B–8B parameter range [Ma et al. \(2025\)](#); [Liu et al. \(2025\)](#). Despite these promises, practical adoption of 1.58-bit models remains limited due to two unresolved challenges. First, most studies report performance

on aggregate or general-knowledge benchmarks, leaving it unclear whether ternary weights can preserve capabilities in complex tasks—such as mathematics, coding, and multi-step reasoning—especially at parameter scales suitable for on-device deployment. Second, production-grade ternary quantization-aware training (QAT) pipelines are almost exclusively built for CUDA/GPU environments, lacking robust implementations on alternative hardware architectures such as domestic Neural Processing Units (NPUs). This software gap restricts the validation and deployment of extreme low-bit methods within a single hardware ecosystem.

To address these limitations, we present a systematic, family-level study of 1.58-bit ternary QAT on the Huawei Ascend NPU. Building upon our prior GPU-based framework, BitCPM4 Team (2025a), we port and optimize the entire training pipeline to the Compute Architecture for Neural Networks (CANN), MindSpeed Huawei Technologies (2024), and Megatron-LM Shoenberger et al. (2019). We train a family of four ternary models—BitCPM-CANN-0.5B/1B/3B/8B—where each model is strictly aligned with its full-precision MiniCPM4 counterpart under identical model architectures and pre-training datasets. This rigorous 1:1 comparison provides a precise and transparent evaluation of the performance impact of low-bit quantization.

Our findings validate the feasibility of scaling ternary LLMs on alternative hardware platforms. Specifically, our contributions include:

- **Preservation of Complex Reasoning Capabilities at Scale:** On reasoning-heavy tasks such as BBH and GSM8K, the 3B and 8B ternary models recover nearly all of their full-precision performance, with the 3B variant achieving parity on BBH. For models with 1B parameters or more, the ternary family retains **95.7%–97.2%** of the full-precision baseline across 11 benchmarks.
- **Analysis of Capacity Constraints in Small-Scale Quantization:** The 0.5B model retains 90.1% of its baseline performance, with the degradation primarily occurring in mathematics. This suggests that for sub-billion parameter models, model capacity—rather than the quantization scheme—is the limiting factor, pointing to data curation and curriculum strategies as more effective avenues for improvement than further quantizer optimization.
- **First End-to-End 1.58-Bit Training System on Ascend NPUs, Lightweight Enough for Default-On Use:** We develop custom ternary operators, integrate them with Megatron-LM and MindSpeed, and validate multi-node distributed training up to the 8B parameter scale on Ascend NPUs. Ternary QAT introduces only a **4.5%** training throughput overhead (148 vs. 155 TFLOP/s per NPU), making it viable as a default training configuration, while enabling an $8\times$ theoretical reduction in weight memory (approximately $6\times$ end-to-end when accounting for scaling factors) to accelerate downstream inference. This implementation closes the infrastructure gap, allowing extreme low-bit techniques to be validated and deployed outside the CUDA ecosystem.

2 Related Work

Low-Bit Quantization for LLMs. Post-training quantization (PTQ) methods, such as GPTQ Frantar et al. (2022) and AWQ Lin et al. (2023), compress pre-trained models to INT4 or INT8 with minimal calibration data. Although effective for moderate compression, PTQ at extreme low bit-widths (e.g., ≤ 2 -bit) often incurs severe accuracy loss. In contrast, quantization-aware training (QAT) Liu et al. (2023) embeds quantization directly into the training loop, allowing the model parameters to adapt to discrete representations and yielding superior quality at ultra-low bit-widths. Recent advances have focused on improving the efficiency of QAT. For instance, EfficientQAT Chen et al. (2024) uses a two-stage block-wise and end-to-end approach to quantize LLaMA-2-70B to 2-bit, while LR-QAT Bondarenko et al. (2024) reduces memory requirements by combining low-rank auxiliary weights with reduced-precision master weights. While those methods focus on lowering training costs, we concentrate on deploying end-to-end 1.58-bit QAT to a domestic NPU hardware ecosystem across a full scale-aligned model family.

1.58-Bit and Ternary Models. BitNet Wang et al. (2023) introduced 1-bit weight training for Transformer architectures, and BitNet b1.58 Ma et al. (2024) extended this approach to ternary weights in $\{-1, 0, 1\}$. Ternary weights allow low-bit models to match full-precision performance at equivalent parameter scales when trained from scratch. Subsequently, BitNet b1.58 2B4T Ma et al. (2025) demonstrated that ternary training scales effectively up to 2B parameters on 4T tokens. ParetoQ Liu et al. (2025) offered a unified framework analyzing trade-offs across 1-bit to 4-bit configurations, scaling 1.58-bit training to an 8B model. Our prior work, BitCPM4 Team (2025a), achieved competitive 1.58-bit results at 0.5B and 1B scales with high training

Table 1: Properties of candidate quantization spaces. “Sym” denotes whether the space is symmetric around zero. “Zero” indicates whether zero is included in the discrete set.

METHOD	VALUES	BITS	SYM	ZERO
TERNARY	3	1.58	✓	✓
ASYM-4-VAL	4	2.00	×	✓
HALF-SYM-4-VAL	4	2.00	✓	×
INT-SYM-4-VAL	4	2.00	✓	×
5-VALUE	5	2.32	✓	✓

efficiency. BitCPM-CANN extends this line of research in two ways: first, we evaluate a complete 0.5B–8B model family directly aligned with same-architecture full-precision baselines, and second, we implement the training system on the Huawei Ascend NPU platform rather than CUDA-enabled GPUs.

Ascend NPU Training Ecosystem. The Huawei Ascend NPU, coupled with the CANN (Compute Architecture for Neural Networks) software stack and the MindSpeed [Huawei Technologies \(2024\)](#) acceleration library, offers a powerful alternative to the CUDA ecosystem for large-scale model training. MindSpeed adapts Megatron-LM [Shoeybi et al. \(2019\)](#) with NPU-specific optimizations such as fused operators, context parallelism, and Token-N-Dim (TND) tensor layouts. This work integrates 1.58-bit QAT into this ecosystem, making low-bit training natively available on Ascend NPUs.

3 Method: Ternary Quantization-Aware Training

3.1 Quantization Space Design

A key design choice in low-bit training is the discrete quantization space—the set of representable values used for projecting latent full-precision weights during the forward pass. This choice determines both the bit budget per weight and the geometric arrangement of the quantized levels, including their symmetry and the inclusion of a zero level. To understand how these factors affect convergence, we perform a controlled ablation across five candidate configurations:

1. **Ternary** (3 values, 1.58 bits/weight): $\{-1, 0, 1\}$ with $s = \text{mean}(|\mathbf{W}_{\text{group}}|)$; symmetric, contains zero.
2. **Asymmetric 4-value** (4 values, 2.00 bits): $\{-2, -1, 0, 1\}$ with $s = \max(|\mathbf{W}_{\text{group}}|)/1.5$; contains zero, asymmetric.
3. **Half-integer symmetric 4-value** (4 values, 2.00 bits): $\{-1.5, -0.5, 0.5, 1.5\}$ with $s = \max(|\mathbf{W}_{\text{group}}|)/1.5$; symmetric, no zero.
4. **Integer symmetric 4-value** (4 values, 2.00 bits): $\{-2, -1, 1, 2\}$ with $s = \max(|\mathbf{W}_{\text{group}}|)/2$; symmetric, no zero.
5. **5-value** (5 values, 2.32 bits): $\{-2, -1, 0, 1, 2\}$ with $s = \max(|\mathbf{W}_{\text{group}}|)/2$; symmetric, contains zero.

Table 1 summarizes the properties of the five candidates and exposes the structure of the sweep: the symmetric/asymmetric and zero/no-zero axes are crossed against three bit budgets (1.58, 2.00, 2.32 bits/weight). The candidates therefore cover both directions in which the ternary baseline could be relaxed—by adding more discrete levels at the same geometric arrangement (the 5-value scheme), or by changing the arrangement at the same level count (the three 4-value schemes).

Experimental protocol. Each variant is fine-tuned from the same 0.5B MiniCPM4 checkpoint for approximately 1.5B tokens using identical training data. Since the scaling factor s is computed directly from weight statistics rather than optimized as a learnable parameter, this setup isolates the impact of the quantization space geometry from other hyperparameters like learning rate, optimizer state, or initialization.

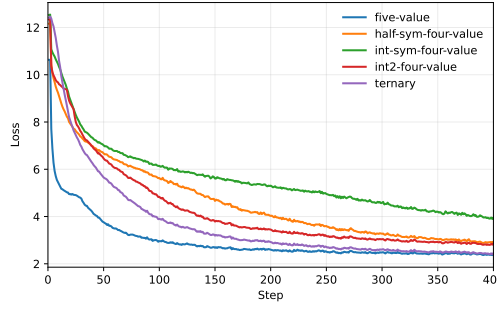


Figure 1: Training loss comparison across five candidate quantization spaces on 0.5B model. Methods satisfying both symmetry and zero-inclusion (ternary, 5-value) achieve fastest convergence.

Results. Figure 1 illustrates the training loss trajectories of the five variants. We make the following observations:

- **Zero inclusion and symmetry are both critical for convergence.** The two configurations that feature both symmetry and a zero level (ternary and 5-value) exhibit the fastest convergence and achieve the lowest final loss. The other three variants converge more slowly, despite some having more quantization levels than ternary.
- **Increasing the number of levels without maintaining symmetry can degrade performance.** The asymmetric 4-value scheme introduces a fourth quantization level but breaks symmetry, resulting in slower convergence than the ternary baseline. This indicates that bit-width alone is not a sufficient predictor of optimization quality when the geometric properties are compromised.
- **The presence of a zero level is highly influential.** The convergence gap between the two symmetric, zero-free variants (half-integer and integer) and ternary is larger than that between the asymmetric zero-containing variant and ternary. This suggests that representing zero explicitly is a crucial property for effective QAT.

The initial loss at the start of QAT also reflects this trend. The 5-value scheme begins with the lowest loss (~ 10.6), whereas the other four schemes start above 12. This is because a denser, symmetric, and zero-containing set introduces less quantization noise during the initial projection of full-precision weights. Over the 1.5B-token training run, the convergence efficiency ranks as follows: 5-value > ternary > integer-symmetric $\approx$ half-integer-symmetric > asymmetric-4-value.

Choice of ternary. We select ternary as the primary quantization space, based on two observations from the sweep.

First, under the weight-statistics-based scaling of our sweep, the gap between ternary and the 2-bit candidates is structural: among four-level grids under a 2-bit budget, none can be both symmetric and contain zero, so every standard 2-bit scheme forfeits at least one of the two properties our sweep identifies as decisive for convergence. Restoring both demands the 5-value grid, whose 47% bit-budget premium (2.32 vs. 1.58 bits/weight) is not justified by the residual loss reduction at deployment.

Second, although 1.58 bits is not natively aligned with the 2/4/8-bit granularities of current accelerators, ternary weights pack cleanly into a 2-bit storage layout and reuse the native 2-bit instructions of recent mobile SoCs (Section 6). Realizing comparable accuracy from a standard 4-level 2-bit format requires deeper hardware–software co-design—e.g., learnable scales and extended QAT Liu et al. (2025)—to compensate for the asymmetry it cannot avoid. Embedding ternary into the 2-bit path is thus the more direct route to today’s 2-bit-capable hardware.

3.2 Ternary Quantizer with STE

Following the choice of ternary quantization, the quantizer maps each weight group to $\{-1, 0, 1\}$ using a group-wise scaling factor, while backpropagating gradients via a straight-through estimator (STE). For a weight tensor $\mathbf{W}$ split into groups of size g , the per-group scale and quantized weights are computed as:

$$s = \max(\text{mean}(|\mathbf{W}_{\text{group}}|), \epsilon), \quad (1)$$

$$\mathbf{W}_q = \text{clamp} \left(\text{round} \left(\frac{\mathbf{W}_{\text{group}}}{s} \right), -1, 1 \right) \cdot s. \quad (2)$$

Two aspects of this design are important: First, the scale s is determined entirely by weight statistics rather than optimized as a learnable parameter. This simplifies training, as the optimizer only updates the latent full-precision weights while the quantization grid adjusts automatically. Second, the mean-of-absolute-value formulation, unlike max-based scaling, allows for local scale computation on each shard in tensor-parallel settings without requiring cross-rank reductions (Section 4.2).

To enable gradient flow through the non-differentiable rounding and clamping operators, we use the straight-through estimator (STE) [Bengio et al. \(2013\)](#) implemented as an identity function with a detached residual:

$$\text{output} = \mathbf{W} + \text{stopgrad}(\mathbf{W}_q - \mathbf{W}). \quad (3)$$

During training, the forward pass utilizes the quantized weight $\mathbf{W}_q$ for downstream operations, while the backward pass computes gradients with respect to $\mathbf{W}$ using the identity mapping. At model export time, the detached residual is discarded, and the model is saved as a native ternary tensor with its corresponding per-group scale, requiring no runtime overhead during inference.

3.3 Two-Stage Decoupled Distillation

Recent scaling-law analyses of low-bit QAT identify a qualitative behavior transition at the 2-bit boundary [Liu et al. \(2025\)](#): for 3-bit and above, QAT acts as a *compensation* process that perturbs weights only within their adjacent quantization cells; for 2-bit and below—including 1.58-bit ternary—QAT enters a *reconstruction* regime in which weights must shift substantially beyond their initial grids to form new semantic representations. This reconstruction phase is intrinsically noisy and data-hungry: the latent full-precision weights move at a much faster pace than in standard fine-tuning, and sub-2-bit QAT saturates only after substantially more tokens than higher-bit QAT.

Layering teacher distillation on top of this reconstruction regime introduces two compounding problems. First, the distillation signal pulls the student toward the teacher’s existing full-precision representation, competing with the reconstruction direction along which ternary QAT must travel; the two gradient sources interfere rather than reinforce, amplifying the variance of the discrete forward pass and occasionally triggering training collapse at small scales. Second, the same interference degrades the token-efficiency of an already data-hungry QAT budget, since gradient updates spent on resolving the conflict do not contribute to either objective.

We therefore propose a *two-stage decoupled training strategy*: we first run pure ternary QAT to convergence, letting the model complete its weight reconstruction unencumbered, and then perform post-training distillation on the stabilized ternary student. This decoupling yields two concrete benefits. First, when distillation begins, the student has completed the reconstruction phase and established a stable ternary weight representation; the distillation gradients then act as a pure refinement signal, rather than competing with an ongoing weight reorganization. Second, separating the two stages allows the QAT and distillation hyperparameters to be tuned independently for their respective objectives.

For teacher selection, we find that teacher performance is more critical than size proximity to the student model. Table 2 compares the distillation of a 0.5B student model using an 8B teacher versus a size-matched 0.5B teacher. The 8B teacher achieves higher scores across the evaluated tasks despite the larger capacity gap. This demonstrates that the teacher’s accuracy and task coverage, rather than structural similarity to the student, dictate the quality of the transferred representation.

¹Scores are from an early-stage training checkpoint used for ablation purposes and are not directly comparable to the final results in Table 3.

Table 2: Effect of teacher quality on 0.5B ternary model distillation. “Avg” is the mean over all 15 evaluated tasks; selected per-task scores are shown for illustration.¹

TEACHER	AVG (15)	MMLU	CMMLU	BBH	GSM8K
NONE	45.21	46.97	59.11	47.97	36.39
0.5B	45.97	45.03	59.96	49.24	37.68
8B	46.23	46.80	59.86	49.36	39.88

A four-layer vertical stack for BitCPM-CANN and quantization-aware training on Ascend NPUs.

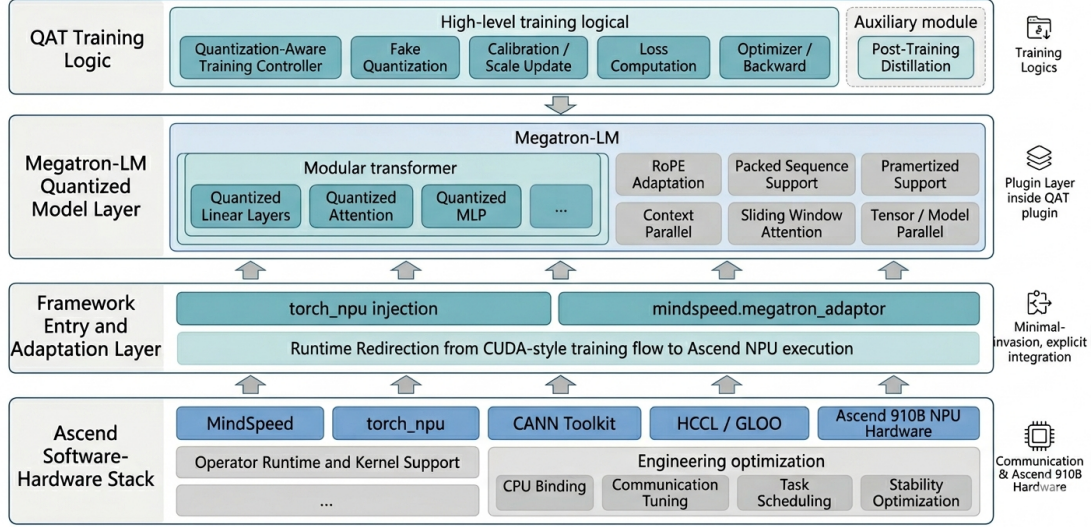


Figure 2: Overall architecture of BitCPM-CANN as a four-layer vertical stack for quantization-aware training on Ascend NPUs. From top to bottom, the figure shows the QAT training logic, the Megatron-LM quantized model layer, the framework entry and adaptation layer with `torch_npu` injection and `mindspeed.megatron_adaptor`, and the Ascend software-hardware stack including MindSpeed, `torch_npu`, CANN, communication backends, and Ascend NPU hardware.

4 Infrastructure: Ascend NPU Adaptation

4.1 Overview

To deploy our system on Ascend NPUs while preserving the training logic, model definition, and data pipelines of our mature CUDA-based Megatron-LM/MindSpeed stack, we adopt a layer-by-layer adaptation strategy. Each layer is modified at its interface with the runtime, ensuring that NPU-specific optimizations are localized while the upper-layer APIs remain unchanged. The adapted stack comprises four layers:

1. **Environment layer:** Migrating from CUDA to the CANN toolkit, mounting drivers, and initializing HCCL/GLOO communication protocols.
2. **Framework entry layer:** Integrating `torch_npu` and `mindspeed.megatron_adaptor` at training entry points, enabling NPU execution without restructuring high-level logic.
3. **Operator and parallelism layer:** Adapting core operations (Attention, RoPE, sliding window attention, TND layout) and Context Parallelism for NPU-specific execution semantics.
4. **Engineering optimization layer:** Applying CPU core binding, optimizing communication configurations, configuring task queues, and improving multi-node training stability.

The top two layers are largely mechanical—porting environment variables, drivers, and entry-point imports—and are handled by upstream `torch_npu` and `mindspeed.megatron_adaptor` injection without changes to the

training logic. The substantive work lies in the bottom two layers, which we address in turn: Section 4.2 describes the operator and parallelism adaptations required to bring ternary QAT, NPU-native attention, and long-sequence training online; Section 4.3 then describes the system-level optimizations that stabilize and accelerate the resulting distributed training jobs.

4.2 Feature-Level Adaptation

This subsection covers the operator- and parallelism-layer adaptations that close three concrete gaps between our CUDA-based training pipeline and the Ascend stack: integrating ternary QAT into Megatron-LM’s tensor-parallel linear layers without breaking communication overlap, supporting the NPU-native TND attention layout for variable-length packed batches, and enabling 32K-sequence training via Ulysses context parallelism coupled with a fused RoPE kernel.

QAT plug-in integration into TP linear layers. We integrate QAT into Megatron-LM’s column- and row-parallel linear layers using three core design choices to minimize QAT overhead on the tensor-parallel (TP) critical path.

(1) Standard STE to preserve TP communication overlap. By implementing the STE (Section 3.2) in the identity-with-detached-residual form, the forward pass computes the quantized weights while the backward pass acts as an identity function. Since the STE does not introduce new communication operations, Megatron-LM’s column-parallel backward path—where the asynchronous `all_reduce` of the activation gradients (`dgrad`) is overlapped with the weight gradient (`wgrad`) computation—remains unaffected. This minimizes the training throughput overhead to only 4.5% (Section 5.3).

(2) Per-shard local weight scaling. Each TP rank computes its scaling factor locally using its local weight shard, $s = \text{mean}(|\mathbf{W}_{\text{group}}|)$, eliminating cross-rank synchronization on the critical path. Because the scaling factor is based on the mean rather than the maximum value, the deviation between local and global scaling factors is negligible in practice, as demonstrated by the strong model performance reported in Section 5.2.

(3) Alignment with the distributed communication graph. In column-parallel layers, weight quantization is performed on the local shard and activation quantization is applied to the distributed `input_parallel` tensor. No quantizer is placed on the `output_parallel` tensor, keeping its precision consistent with subsequent layers. In row-parallel layers, the activation quantizer is applied to the local `input_parallel` tensor *before* the `reduce_scatter` operation, allowing the quantized inputs to feed the local GEMM while the cross-rank reduction behaves exactly as in standard Megatron-LM.

The quantization configuration (e.g., bit-width, group size) is managed via a unified configuration file, enabling seamless switching between low-bit and full-precision modes.

TND mode and packed sequence support. The fused attention operators on Ascend NPUs adopt a Token-N-Dim (TND) layout, where tokens within a packed micro-batch are flattened into a single axis alongside cumulative sequence lengths. This differs from the Sequence-Batch-Head (SBH) layout typically assumed by Megatron-LM on GPUs. Since the two layouts follow incompatible indexing and padding conventions, they cannot be reconciled through simple transpositions; we therefore introduce a dual-path execution scheme that dispatches between SBH and TND at runtime based on the presence of `packed_seq_params`. When the TND path is active, the cumulative sequence length tensors (`cu_seqLens_q/kv`) are reformatted to match the conventions of the NPU fused attention kernels, and activations are reshaped between the $[T, 1, N, D]$ and $[T, N, D]$ formats via lightweight `squeeze/unsqueeze` operations placed immediately around each attention call. Under `use_span_based_attn` mode, dedicated patches are further applied to the RoPE and attention layers to enforce consistent treatment of intra-sequence boundaries. The fused kernel is then invoked with `shape_order="TND"`, enabling variable-length attention without padding shorter sequences to a common length.

Context parallel and long-sequence training. For long sequences (e.g., 32K tokens), single-rank execution faces severe memory constraints due to the large activation footprint, and rotary position embeddings (RoPE) introduce notable computational overhead. To mitigate these bottlenecks, we combine Context Parallelism (CP) along the sequence dimension with the TND attention path and utilize a fused RoPE operator. Specifically,

Ulysses CP partitions sequences across the CP dimension with MindSpeed managing the underlying all-to-all communication; a *custom fused RoPE operator* performs rotary position embedding with native multi-batch reshaping on TND tensors, applying the rotation and restoring the layout in a single pass without allocating intermediate buffers; and *multi-micro-batch support* is enabled for configurations with a micro-batch size (MBS) ≥ 2 by reconstructing position IDs from `packed_seq_params`.

4.3 Engineering Optimizations

Beyond bringing the above features online, sustaining stable long-sequence training at scale requires addressing system-level bottlenecks that do not typically appear in single-node or short-context setups. These include host-side scheduling jitter that desynchronizes training ranks, launch overheads between sequential kernels, network congestion from co-located traffic, and sub-optimal operator performance under long-context workloads. We address these issues through four key optimizations:

CPU core binding for host-side determinism. Without thread affinity, NPU task submission threads and PyTorch worker threads migrate freely across CPU cores. The resulting scheduling jitter is negligible on a single node but accumulates in distributed training, where each step is paced by the slowest rank. We pin host threads to dedicated CPU cores, bounding host-side latency variance and reducing inter-rank desynchronization.

NPU task pipelining via `TASK_QUEUE_ENABLE=2`. We increase the NPU task queue depth by setting `TASK_QUEUE_ENABLE=2`, which allows the host to enqueue subsequent kernels while the device is executing active tasks. This pipelining prevents host-side latencies from introducing idle gaps on the device. This optimization is particularly beneficial for the dense kernel sequences in the attention and MLP blocks, where kernel launch overheads would otherwise serialize and degrade the benefits of operator fusion.

Explicit NIC selection for HCCL. In multi-node configurations, the HCCL collective communications required by tensor-parallel backward passes (Section 4.2) and context-parallel all-to-all operations (Section 4) can be routed over network interface cards (NICs) shared with other system traffic. This sharing introduces communication-time jitter. By pinning HCCL to dedicated NICs at job launch, we ensure that cross-node communication uses deterministic, isolated network paths, stabilizing step times.

Defaulting to span-based attention and fused RoPE. We make the variable-length attention and fused RoPE paths the default rather than optional. Span-based attention avoids padding shorter sequences in packed batches, and the fused RoPE kernel collapses the reshape-rotate-reshape sequence into a single operation. Enforcing these optimized routines prevents performance regressions from slower, fallback implementations and ensures high throughput during long-context training.

These optimizations enable stable, long-term training on a 2-node, 16-NPU Ascend cluster at a 32K sequence length using the CP and TND attention paths. The system achieves a training throughput of approximately 2,700 tokens/s per NPU for the 3B model and 1,340 tokens/s per NPU for the 8B model. These system-level adjustments only optimize execution timing and hardware utilization without altering the numerical outputs of the training steps; thus, the accuracy and throughput results are collected from identical execution runs.

5 Experiments

5.1 Setup

Models. We train four BitCPM-CANN models of sizes 0.5B, 1B, 3B, and 8B. Each model is initialized from the corresponding full-precision MiniCPM4 checkpoint and optimized using our two-stage pipeline: ternary QAT to convergence followed by post-training distillation.

Table 3: Main results: BitCPM-CANN vs. MiniCPM4 full-precision baselines across 11 benchmarks grouped into three categories. “Avg” denotes the mean score across all 11 tasks. “Ret.” is the retention ratio.

Category	Task	8B		3B		1B		0.5B	
		FP	Ternary	FP	Ternary	FP	Ternary	FP	Ternary
Commonsense	ARC-c	87.46	86.10	80.34	78.98	64.41	67.12	51.86	50.51
	ARC-e	95.06	93.47	92.77	88.36	79.89	79.01	71.78	65.08
	BoolQ	84.89	83.39	79.85	77.89	68.38	65.50	62.29	43.55
	PIQA	80.52	78.78	70.57	72.69	66.16	65.45	60.99	58.49
	WinoGrande	63.30	61.17	58.41	52.96	51.62	53.28	51.07	51.54
Domain Knowledge	CMMLU	80.62	78.92	78.11	76.53	74.57	67.42	65.22	60.49
	C-Eval	81.36	77.50	75.85	75.89	73.25	65.96	66.11	60.74
	MMLU(5shot)	75.83	70.65	66.95	64.41	57.71	57.71	55.55	50.73
	MMLU-Redux	77.14	69.85	65.82	60.07	54.80	54.16	48.00	43.79
Math & Reasoning	BBH	76.72	70.70	68.29	68.30	64.40	60.40	49.87	47.44
	GSM8K	91.51	85.75	81.64	79.45	63.15	61.56	52.08	39.42
Average (11 tasks)		81.31	77.84	74.42	72.32	65.30	63.42	57.71	51.98
Retention		95.7%		97.2%		97.1%		90.1%	

Baselines. Each BitCPM-CANN model is compared against its full-precision MiniCPM4 baseline of the same size. This ensures that the only difference between the compared models in Table 3 is the use of ternary linear weights in place of BF16 weights.

Benchmarks. We evaluate performance on 11 tasks categorized into three areas:

- *Commonsense reasoning:* ARC-Challenge, ARC-Easy, BoolQ, PIQA, WinoGrande.
- *Domain knowledge:* CMMLU, C-Eval, MMLU, MMLU-Redux.
- *Mathematics & reasoning:* BBH, GSM8K.

Hardware. All training runs are executed on multi-node Huawei Ascend NPU clusters using the adapted Megatron-LM and MindSpeed stacks described in Section 4.

5.2 Main Results

We make the following observations:

- **Models of 1B and larger achieve $\geq 95.7\%$ retention.** The 3B model achieves the highest retention at 97.2%, followed by the 1B (97.1%) and 8B (95.7%) models. At these scales, ternary QAT causes minimal performance degradation, even on complex mathematical and multi-step reasoning benchmarks.
- **The 0.5B model exhibits higher sensitivity to quantization.** The smallest model retains 90.1% of the baseline performance, with the degradation concentrated in reasoning-heavy tasks. This indicates that quantization noise is more challenging to absorb in models with limited capacity, suggesting that curriculum learning or specialized training data may be more effective than further quantizer tuning at smaller scales.
- **Direct alignment with MiniCPM4.** Because each BitCPM-CANN model is evaluated alongside its full-precision counterpart using identical architectures and datasets, the results provide a direct reference for evaluating deployment trade-offs when substituting BF16 models with ternary variants.

Across the model scales, the retention curve (90.1% for 0.5B, 97.1% for 1B, 97.2% for 3B, and 95.7% for 8B) exhibits a significant increase from 0.5B to 1B, followed by stable performance at larger sizes. Once a model reaches 1B parameters, ternary QAT consistently preserves baseline performance within approximately 5% on average.

Table 4: Training throughput comparison on Ascend NPU.

CONFIGURATION	TFLOP/s PER NPU	OVERHEAD
FULL-PRECISION	155	—
TERNARY QAT	148	4.5%

Table 5: Distillation strategy comparison on 0.5B model.²

STRATEGY	AVG SCORE
PURE QAT (NO DISTILLATION)	43.86
DISTILLATION THROUGHOUT QAT	44.91
TWO-STAGE DECOUPLED DISTILLATION (OURS)	45.95

5.3 Training Efficiency

To assess whether QAT can be enabled by default during training, we evaluate its throughput overhead compared to the full-precision BF16 baseline. Table 4 measures this performance on the Ascend NPU using identical model configurations, parallelism settings, and sequence lengths, isolating the computational cost of the ternary quantizers.

Ternary QAT introduces only a 4.5% throughput overhead. As discussed in Section 4.2, this minimal cost is achieved because the identity-residual STE avoids introducing additional communication collectives, preserving Megatron-LM’s native overlap between the TP dgrad communication and wgrad computations. The remaining overhead stems entirely from local element-wise calculations (group reductions, scale computations, and rounding operations), keeping QAT off the training critical path.

For context, under the 32K CP+TND context-parallel setting described in Section 4, the 2-node, 16-NPU Ascend cluster achieves:

- 3B model: $\sim 2,700$ tokens/s per NPU.
- 8B model: $\sim 1,340$ tokens/s per NPU.

5.4 Ablation Studies

Distillation Strategy. We evaluate three distillation regimes on the 0.5B ternary model: (1) Pure QAT (no distillation), (2) Distillation throughout QAT, which enables the distillation loss from the start of QAT and runs as a single training stage, and (3) Two-stage decoupled distillation (Ours), which runs QAT first to obtain a converged ternary student, and then performs distillation.

Table 5 presents the comparison results. Pure QAT obtains an average score of 43.86. Enabling distillation throughout QAT slightly improves the score to 44.91, but we observe that this single-stage co-optimization can cause instability, as the intense gradient variance from early QAT limits the model from absorbing the teacher’s knowledge. In contrast, our Two-stage decoupled distillation yields a significant boost to 45.95. This confirms that decoupling the two phases is highly beneficial: stabilizing the student’s ternary weight grid first allows the subsequent distillation process to focus purely on prediction refinement rather than struggling with an ongoing weight reconstruction.

Learning Rate Sensitivity. Ternary QAT exhibits higher sensitivity to learning rates than standard full-precision training. We evaluate this sensitivity using the 0.5B model, as detailed in Table 6.

While the ternary model benefits from a moderate learning rate of 1×10^{-2} (improving training loss and final benchmark scores over 5×10^{-3}), the effective range is highly restricted. Raising the learning rate to 1.5×10^{-2} results in higher stable-phase loss and prevents successful loss recovery during the decay

²Scores are from an early-stage training checkpoint used for ablation purposes and are not directly comparable to the final results in Table 3.

Table 6: Effect of learning rate on 0.5B ternary QAT. Loss values reported at the end of stable and decay phases.

LR	STABLE LOSS	DECAY LOSS	SCORE
5E-3	2.26	1.661	45.35
1E-2	2.24	1.646	46.74
1.5E-2	2.27	—	DEGRADED

Table 7: Effect of quantization group size on 0.5B model.

GROUP SIZE	STABLE LOSS	DECAY LOSS
PER-CHANNEL ($g = -1$)	2.26	1.661
$g = 64$	2.24	1.646
$g = 32$	2.232	1.635

phase, leading to model degradation. This highlights the importance of optimizing the QAT learning rate independently rather than inheriting configurations from full-precision baselines.

Group Size. The group size g (Section 3.2) defines the granularity of the scaling factor relative to the weight distribution. Per-channel scaling ($g = -1$) uses a single scale factor for an entire row, whereas smaller group sizes compute local factors at sub-row granularity. Table 7 sweeps this hyperparameter on the 0.5B model.

Reducing the group size from per-channel to $g = 64$ and $g = 32$ leads to consistent reductions in both stable-phase and decay-phase losses, with $g = 32$ yielding the best performance. Although the absolute improvement is modest, the trend is robust, indicating that group size is a significant hyperparameter affecting model convergence.

Math Task Sensitivity at Small Scale. Evaluating the reasoning-heavy tasks (GSM8K and BBH) in Table 3 reveals an asymmetric degradation pattern across scales. For the 0.5B model, GSM8K performance drops from 52.08 to 39.42 and BBH from 49.87 to 47.44. Conversely, the 3B and 8B models recover almost all of their full-precision performance, with the 3B model achieving parity on BBH. This suggests that absorbing quantization noise requires sufficient model capacity, which smaller models lack. Consequently, improving small-scale performance may depend on data strategies, such as customized curriculum learning or adjusted reasoning-data ratios, rather than modifications to the quantizer itself.

6 Future Work

We identify four key directions for future research:

1. **Improving small-scale retention.** The 90.1% retention of the 0.5B model, especially on reasoning benchmarks, indicates that future gains at small scales will likely stem from advanced data mixtures, curriculum learning, or task-specific continued training rather than quantizer modifications.
2. **Extending to MoE architectures.** Mixture-of-Experts (MoE) models represent a promising target for ternary QAT. Applying our pipeline to MoE architectures could significantly reduce expert memory footprints. Combined with expert offloading, ternary weights could enable 30B-class MoE models, such as Qwen3-30B-A3B Team (2025b), to fit within a 4 GB on-device memory budget, facilitating edge deployment.
3. **Optimizing inference for 2-bit hardware.** While our training overhead is minimal (4.5%), realizing the theoretical $8\times$ weight memory reduction during deployment requires optimized W2A8 operators and packed formats. Recent mobile processors (e.g., Qualcomm Snapdragon 8850 and 8397) feature native 2-bit inference instructions, making the development of highly optimized ternary runtimes highly practical.

4. **Full-stack low-bit training.** Extending ternary QAT from the pretraining and SFT phases to reinforcement learning from human feedback (RLHF) represents a natural progression toward an entirely low-bit training pipeline.

7 Conclusion

We introduced BitCPM-CANN, an end-to-end 1.58-bit training system natively implemented on Huawei Ascend NPUs. By integrating ternary QAT into Megatron-LM and MindSpeed, we trained a model family (0.5B to 8B) that preserves 90.1%–97.2% of full-precision performance across 11 benchmarks. The system maintains low training overhead (4.5%) while enabling up to an $8\times$ weight memory reduction at inference time.

In addition to demonstrating the feasibility of low-bit scaling, BitCPM-CANN provides a robust, reusable infrastructure for low-bit training on Ascend hardware. The unified checkpoint system, modular quantization layers, and support for long-sequence training (32K via CP+TND) establish a foundation for future low-bit optimization on domestic NPUs.

Acknowledgements

This work is jointly conducted by the High Performance Computing Institute of Tsinghua University, the THUNLP Lab, and ModelBest Inc., with support from Huawei HiSilicon.

References

Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. In *arXiv preprint arXiv:1308.3432*, 2013.

Yelysei Bondarenko, Riccardo Del Chiaro, and Markus Nagel. LR-QAT: Low-rank quantization-aware training for LLMs. *arXiv preprint arXiv:2406.06385*, 2024.

Mengzhao Chen, Wenqi Shao, Peng Xu, Jiahao Wang, Peng Gao, Kaipeng Zhang, Yu Qiao, and Ping Luo. EfficientQAT: Efficient quantization-aware training for large language models. *arXiv preprint arXiv:2407.11062*, 2024.

Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022.

Huawei Technologies. MindSpeed: Accelerating large model training on Ascend NPU. <https://gitee.com/ascend/MindSpeed>, 2024.

IDC. Global Memory Shortage Crisis: Market Analysis and the Potential Impact on the Smartphone and PC Markets in 2026. <https://www.idc.com/resource-center/blog/global-memory-shortage-crisis-market-analysis-and-the-potential-impact-on-the-smartphone-and-pc-markets-in-2026>. IDC Resource Center.

Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Xingyu Dang, and Song Han. AWQ: Activation-aware weight quantization for LLM compression and acceleration. *arXiv preprint arXiv:2306.00978*, 2023.

Zechun Liu, Barlas Oguz, Changsheng Zhao, Ernie Chang, Pierre Stock, Yashar Mehdad, Yangyang Shi, Raghuraman Krishnamoorthi, and Vikas Chandra. LLM-QAT: Data-free quantization aware training for large language models. *arXiv preprint arXiv:2305.17888*, 2023.

Zechun Liu, Changsheng Zhao, Hanxian Huang, Sijia Chen, Jing Zhang, Jiawei Zhao, Scott Roy, Lisa Jin, Yunyang Xiong, Yangyang Shi, et al. ParetoQ: Scaling laws in extremely low-bit LLM quantization. *arXiv preprint arXiv:2502.02631*, 2025.

- Shuming Ma, Hongyu Wang, Lingxiao Ma, Lei Wang, Wenhui Wang, Shaohan Huang, Li Dong, Ruiping Wang, Jilong Xue, and Furu Wei. The era of 1-bit LLMs: All large language models are in 1.58 bits. *arXiv preprint arXiv:2402.17764*, 2024.
- Shuming Ma, Hongyu Wang, Shaohan Huang, Xingxing Zhang, Ying Hu, Ting Song, Yan Xia, and Furu Wei. BitNet b1.58 2B4T technical report. *arXiv preprint arXiv:2504.12285*, 2025.
- Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- MiniCPM Team. MiniCPM4: Ultra-efficient LLMs on end devices. *arXiv preprint arXiv:2506.07900*, 2025a.
- Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025b.
- TrendForce. AI Server Demand to Drive Memory Contract Price Increases in 2Q26 as CSPs Secure Supply via Long-Term Agreements. <https://www.trendforce.com/presscenter/news/20260331-12995.html>, 3 2026. Press release.
- Hongyu Wang, Shuming Ma, Li Dong, Shaohan Huang, Huaijie Wang, Lingxiao Ma, Fan Yang, Ruiping Wang, Yi Wu, and Furu Wei. BitNet: Scaling 1-bit transformers for large language models. *arXiv preprint arXiv:2310.11453*, 2023.